Department of Computer Science & Software Eng.
Concordia University

COMP 6651- Winter 2024

Algorithm Design Techniques

PROJECT

Submitted to:

Dr. Thomas G. Fevens

NAME	ENROLLMENT ID
Muqaddaspreet Singh Bhatia	40276333

Table of Contents

1. PROBLEM DESCRIPTION

The project is focused on Exploring various graph algorithms to calculate the Largest Simple path for the given undirected unweighted graphs. The LSP (Longest Simple Path) is classified as an NP-hard problem.

The primary objective of this project is to implement and analyze various heuristic algorithms to approximate the longest simple path in geometrics and online graphs.

The project includes of following steps:

1.Geometric Graph Generator Development

The Geometric Graph Generator produces undirected, unweighted geometric graphs with nodes randomly placed within a unit square. The connectivity of each graph is determined by a given radius r, and the graphs are saved using the ASCII-based EDGES format.

2.Graph Selection and Preparation

The project will utilize six graphs: three from our custom Euclidean graph generator and three from online repositories. This approach allows us to explore and analyze both synthetic and real-world network structures.

3.Analysis of Graph Components

For each graph, the largest connected component (LCC) is identified and analyzed. These LCCs (Largest Connected Component) form the focus of our algorithmic evaluations, ensuring our analysis is centered on the principal segment of each graph.

4.Implementation of Pathfinding Heuristics

We will implement and test four heuristic algorithms to find the longest simple path in each graph's LCC. This includes three existing methods and one newly designed algorithm, providing a basis for comparative effectiveness analysis.

5.Metric Compilation and Evaluation

Results from the heuristic algorithms will be carefully recorded and analyzed with emphasis on performance metrics. These findings will be organized in tables to highlight each algorithm’s ability to detect the longest paths in complex networks.

2. IMPLEMENTATION:

1.Random Euclidean Generator:

1.Geometric Graph Generation:

Generate_geometric_graph (n, r): Constructs a graph with n nodes randomly placed within a unit square, connecting nodes that are within a distance r of each other.

2. Depth-First Search (DFS):

Dfs (graph, start): Executes a depth-first search from start, using a stack to explore and a set to track visited nodes.

3. Largest Connected Component (LCC):

find_lcc (vertices, edges): Utilizes DFS to determine and return the largest connected component of the graph, along with its size.

4.Graph Storage:

save_graph (filename, vertices, edges): Saves the graph in EDGES format with positional data to a file, facilitating further analysis and visualization.

5. Execution Logic:

main (): Manages graph generation with specific r values to achieve target LCC sizes, saving graphs that meet these criteria.

6. Objective:

The Random Euclidean Generator is implemented through the function **generate_geometric_graph (n, r**) [1], which is responsible for creating a geometric graph. This function focuses on generating undirected, unweighted graphs within a geometric plane.

Node Placement: It places n nodes randomly within a unit square (0 to 1 on the x and y axes). Each node receives random coordinates generated using random. Uniform (0, 1) for both x and y values.

Edge Formation: Edges are formed based on the Euclidean distance between nodes. Using math.dist(), the function checks if the distance between any two nodes is less than or equal to a specified radius **r**. If true, an edge is added between these two nodes, indicating their connectivity within the graph.

The Three Generated graphs satisfies the constraints mentioned in Description:

Graph Name	Nodes	r	LCC constraint	LCC
graph_n300	300	0.0773	$0.9 \cdot n \leq \|VLCC\| \leq 0.95 \cdot n$	270<|**282**|<285
graph_n400	400	0.0663	$0.8 \cdot n \leq \|VLCC\| \leq 0.9 \cdot n$	320<|**358**|<360
graph_n500	500	0.0580	$0.7 \cdot n \leq \|VLCC\| \leq 0.8 \cdot n$	350<|393|<400

The Edges file generated stores the data in ASCII readable format:

V1 V1_x_cordinates V1_y_cordinates V2 V2_x_cordinates V2_y_cordinates

2. Largest Connected Component (LCC)

For LCC Implementation, we Compute the Largest connected component for each graph in the respective implementation of the algorithm, as LCC is required by each Algorithm to calculate LSP.

To calculate the LCC we use the visited set to keep track of all nodes that have been explored. A largest_component list to store the nodes of the largest connected component found during the traversal.

Traversal Setup:

For each node in the graph, if it has not been visited, initiate a DFS to explore all reachable nodes from that node.

Depth-First Search (DFS):

Start from the current node, mark it as visited, and explore all adjacent unvisited nodes recursively.

Accumulate all nodes reached in the current traversal in a list called component.

Component Comparison:

After each component is fully explored, compare its size to the size of the largest_component.

If the current component is larger, update largest_component with the current component.

Output:

Return the largest_component, which will contain all nodes in the largest connected subset of the graph.

Key Functions:

DFS Function: A helper function that takes a node, the graph, a visited set, and a current component list as arguments. It recursively explores all reachable nodes, marking them as visited and adding them to the current component list.

Find_LCC Function: Initiates the DFS from each unvisited node and updates the largest_component based on the size of newly found components.

3. Algorithms:

1. DFS-Based Longest Simple Path:

Input: Graph file in. edges in two of given format

1. node1 x1 y1 node2 x2 y2 for geometric graphs
2. node1 node2

Output: Longest simple path and various LCC metrics

1. Define a Graph class:

- Initialize with a graph structure and a set of vertices.
- Add edge method: Adds an edge to the graph and updates the vertices set.

2. Define methods within the Graph class:

- Get the largest connected component:
- Use a BFS algorithm to traverse and find the largest connected component by counting nodes and edges.
- Calculate degree for a vertex.
- Find the maximum degree among a set of vertices.
- Calculate the average degree given vertices and edge count.

3. Define a DFS function:

- Traverse the graph starting from a given vertex using a stack.
- Track visited vertices and their depth to find the deepest vertex.

4. Define a function to find the longest simple path using DFS:

- Repeatedly execute DFS starting from random vertices to find the deepest point and measure path length.

- Determine the longest path found after several iterations.

5. Define a function to read a graph from a file:

- Read edges from a file and add them to the graph.

6. Main execution logic:

- Read graph from a file.

- Calculate the largest connected component.

- Calculate metrics like number of nodes in LCC, maximum degree, and average degree.

- Use a heuristic to find the longest simple path.

- Print all calculated metrics.

2. Dijkstra's Algorithm for Largest Simple Path:

This modified version of Dijkstra's algorithm is used to find the longest path in terms of the number of edges, rather than the shortest path typically sought with Dijkstra's [1].

Input: Graph file in. edges in two of given format

1. node1 x1 y1 node2 x2 y2 for geometric graphs

2. node1 node2

Output: Longest simple path and various LCC metrics.

Pseudocode:
Algorithm: Longest Simple Path in Largest Connected Component

1. Read Graph:

- Initialize an empty dictionary "edges"

- For each line in the graph file:

- Parse nodes u, v, and their coordinates

- Add v to the adjacency list of u and vice versa

2. Depth-First Search (DFS):

- Initialize "visited" as an empty set

- Use a stack to explore nodes starting from "start"

- Mark each visited node and explore its neighbours that are not visited

Depth-First Search (DFS): The **dfs function** is a standard implementation of depth-first search that traverses the graph from a given start node, marking nodes as visited. It is used to explore each node's connections thoroughly.

3. Find Largest Connected Component (LCC):

- Initialize "visited" as an empty set

- For each vertex in "edges":

  - If vertex is not visited, perform DFS to find all connected nodes (component)

  - Update LCC if the size of the current component is larger than the existing LCC

4. Calculate Degrees:

- For each node in LCC, calculate its degree based on its neighbours in "edges"

- Calculate and return the maximum and average degree of nodes in LCC

5. Modified Dijkstra for Maximum Distance (Dijkstra_Max):

- Initialize "distances" with negative infinity for all nodes except start node (zero)

- Use a max-heap to manage and explore nodes based on longest distances

- Update the distance for each neighbour if a longer path is found through the current node

dijkstra_max: a variation of Dijkstra's algorithm that finds the maximum distance from a starting node to all other nodes within the graph. Unlike the traditional Dijkstra's, which finds the shortest path, this implementation uses a max-heap and keeps track of the longest distances.

6. Find Endpoint of Longest Path in LCC:

- Identify subgraph of nodes and edges belonging to LCC

- For each node in the subgraph:

  - Use Dijkstra_Max to find the farthest reachable node and its distance

  - Track the maximum distance found and corresponding node

find_longest_path_end uses the modified Dijkstra's algorithm to find the endpoint of the longest path from each node in the LCC. It computes this for all nodes to determine the overall longest path within the LCC.

7. Main Execution:

- Load graph data from file

- Find LCC and calculate degrees

- Determine the endpoint of the longest simple path in LCC

- Output size of LCC, maximum degree, average degree, and length of the longest path

3. A* Algorithm for Longest Simple Path:

Algorithm A* algorithm for longest simple path

Input: Graph file in. edges in format **node1 x1 y1 node2 x2 y2**

Output: Longest simple path and various LCC metrics.

1. Define a function to read a graph from an edges file:

- Initialize an empty graph and position dictionary.

- Read each line from the file, extract node information, and update the graph and positions.

2. Define a function to find the largest connected component (LCC) in the graph:

- Use a depth-first search (DFS) approach with a stack to explore nodes.

- Track visited nodes and maintain a list of nodes in the current component.

- Update the largest component found so far based on size.

3. Define a function to calculate degree metrics for the LCC:

- Calculate the maximum and average degree of nodes within the LCC.

4. Define a heuristic function (Euclidean distance) for the A* algorithm [2]:

- **Calculate the Euclidean distance between two points**.

5. Define a function for the A* heuristic search to find the longest simple path:

- Use a priority queue (heap) to maintain nodes to be explored.

- Track visited nodes and their path lengths.

- For each node, explore its neighbors. If a longer path is found, update the priority queue.

- Return the longest path found between a start and a goal node.

6. Define a function to compute metrics using multithreading:

- Calculate degree metrics and use the largest connected component for path calculations.

- Pair up nodes and compute the length of the longest simple path between each pair using A*[2].

- Use a ThreadPoolExecutor to parallelize these calculations.

- Gather results and compute the maximum path length.

7. Load graph and positions from a file.

8. Compute metrics using the multithreaded function and print results.

4. Our heuristic for longest Simple Path:

Input: Graph file in. edges in format **node1 node2**

Output: Longest simple path and various LCC metrics.

1. Define a Graph class:

- Initialize with a graph structure and a set of vertices.

- Add edge method: Adds an edge to the graph and updates the vertices set.

2. Define methods within the Graph class:

- DFS (Depth First Search):

- Traverse the graph using a stack to keep track of nodes.

- Maintain a visited set and a list for the current component.

- Get the largest connected component (LCC):

- Iterate over all vertices, perform DFS from unvisited nodes, and find the largest component.

- Calculate degree metrics (maximum and average) for a given set of vertices.

3. Define a heuristic function:

- **The heuristic could be a function of the negative degree of the node to promote exploration.**

4. Define the A* algorithm for the longest simple path:

- Utilize a priority queue (heap) to manage exploration based on path costs adjusted by the heuristic.

- Track visited nodes and their best path length.

- Continuously extend paths and update the heap with potential paths to explore.

5. Define a function to read a graph from a file:

   - Parse a file to extract edges and build the graph using the add edge method.

6. Main execution logic:

   - Load the graph from a file.

   - Determine the largest connected component (LCC).

   - Calculate maximum and average degree for nodes in the LCC.

   - Apply the A* algorithm from an arbitrary start node in the LCC to find the longest path.

   - Print the computed metrics such as the number of nodes in the LCC, maximum degree, average degree, and length of the longest path.

The only difference between Modified and our Heuristic Algorithm is implementation of Heuristic Function. The Modified A* Algorithm uses the Euclidean Distance between two points as a heuristic whereas we use negative degree of the node to promote the exploration.

3. CORRECTNESS:

Testing on smaller Dataset:

To evaluate the correctness of our algorithm, we evaluate the algorithms on smaller datasets that are easier to visualize, and metrics calculation is relatively easier.

Table 1: Results table for random graph with n = 10

Algorithm	Lmax (LSP)	VLCC	Δ(LCC)	k(LCC)
DFS-heuristic	4	4	4	3
Dijkstra's-heuristic	4	4	4	3
A* Star	3	4	4	3
Our Heuristic	4	4	4	3

Table 2: Results table for random graph with n = 5

Algorithm	Lmax (LSP)	VLCC	Δ(LCC)	k(LCC)
DFS-heuristic	2	2	2	2
Dijkstra's-heuristic	2	2	2	2
A* Star	2	2	2	2
Our Heuristic	2	2	2	2

1. Consistency Across Graph Sizes: Notice how the algorithms perform as the graph size changes from n=5 and n=10.Algorithms showing less variation in metrics as graph sizes change might be more robust or scalable.

2. Algorithm Performance: For each metric, compare the results of each algorithm. Algorithms that consistently have lower values for Lmax or higher values for other metrics are considered more effective.

To prove correctness of the Algorithms:

1. **Depth-First Search (DFS)-Based Algorithm for Longest Simple Path**

Correctness of DFS for Traversal:

DFS Mechanism: DFS explores as far as possible along each branch before backtracking. This property is useful in finding the longest simple path because it naturally explores deep paths.

Handling of Cycles: By maintaining a visited set, DFS avoids revisiting nodes, which ensures that paths considered are simple (no repeated vertices).

Correctness of Finding the Longest Path using DFS:

DFS is repeatedly executed from various vertices to find the deepest points. While DFS is not guaranteed to find the longest simple path in all graph types due to its greedy nature in depth exploration, random restarts increase the chance of finding a longer path, especially in sparse graphs.

2. **Dijkstra's Algorithm Modified for Longest Path**

Correctness of Modified Dijkstra's Algorithm:

Max-Heap Utilization: Traditional Dijkstra's algorithm uses a priority queue to explore the shortest paths first. The modification uses a max heap to prioritize the longest paths. This adaptation is crucial in ensuring that the longest paths are explored preferentially.

Update Criterion: The distance update criterion is inverted to consider longer paths rather than shorter ones, which is a correct approach for maximizing path length.

3. **A* Algorithm for Longest Simple Path**

Correctness of A* Algorithm:

Heuristic Function: If the heuristic is admissible (never overestimates the true path cost) and consistent (monotonically increasing), the A* algorithm is guaranteed to find the longest path first among all paths leading to the goal node. However, for longest path problems, the challenge lies in defining such a heuristic.

Pathfinding Guarantee: The A* algorithm, with a suitable heuristic, will find the longest simple path if the path costs (weights) and heuristic allow for exploration prioritization based on path length rather than traditional cost minimization.

4. **Our Heuristic**

Correctness of Heuristic Approach:

Algorithm Design: The description involves a heuristic that encourages exploration of underexplored nodes or paths, its correctness in finding the longest path would similarly depend on the specific properties of the heuristic where we can allocate negative degree to the function of the node as a heuristic function.

Integration with A*: If the heuristic is designed to guide search towards deeper (or less commonly visited) parts of the graph effectively, and if it integrates well with DFS or A* principles, then the algorithm be correct in its explorative strategy.

4. RESULTS:

Geometric Graphs:

Table 3: Results table for random graph with n = 300

Algorithm	n	r	|VLCC|	Δ(LCC)	k(LCC)	Lmax
DFS heuristic	300	0.0773	282	24	11.2624113 4751773	133
Dijkstra heuristic	300	0.0773	282	24	11.2624113 4751773	132
A* heuristic	300	0.0773	282	24	11.2624113 4751773	118
Your heuristic	300	0.0773	282	24	11.2624113 4751773	135

Table 4: Results table for random graph with n=400

Algorithm	n	r	|VLCC|	Δ(LCC)	k(LCC)	Lmax
DFS heuristic	400	0.0663	358	30	11.4301675 97765363	173

Dijkstra heuristic	400	0.0663	358	30	11.430167597765363	143
A* heuristic	400	0.0663	358	30	11.430167597765363	133
Your heuristic	400	0.0663	358	30	11.430167597765363	155

Table 5: Results table for random graph with n = 500

Algorithm	n	r	|VLCC|	Δ(LCC)	k(LCC)	Lmax
DFS heuristic	500	0.0580	393	20	10.585241730279899	174
Dijkstra heuristic	500	0.0580	393	20	10.585241730279899	157
A* heuristic	500	0.0580	393	20	10.585241730279899	137
Your heuristic	500	0.0580	393	20	10.585241730279899	133

Online Graphs:

Table 6: Results table for DSJC500-5 graph.

Algorithm	n	r	|VLCC|	Δ(LCC)	k(LCC)	Lmax
DFS heuristic	500	_	500	286	250.496	499
Dijkstra heuristic	500	_	500	286	250.496	499
Your heuristic	500	_	500	286	250.496	499

Table 7: Results table for Euroroad graph.

Algorithm	n	r	|VLCC|	Δ(LCC)	k(LCC)	Lmax

DFS heuristic	1174	–	1039	10	2.512030798845043	320
Dijkstra heuristic	1174	–	1039	10	2.512030798845043	320
Your heuristic	1174	–	1039	10	2.512030798845043	290

Table 8: Results table for power graph.

Algorithm	n	r	|VLCC|	Δ(LCC)	k(LCC)	Lmax
DFS heuristic	4941	–	4941	19	2.66909532483303	1038
Dijkstra heuristic	4941	–	4941	19	2.66909532483303	1038
Your heuristic	4941	–	4941	19	2.66909532483303	511

5. CONCLUSION:

The results presented across six tables comparing the performance of different heuristic algorithms (DFS, Dijkstra, A*, and a custom heuristic) on graphs of various sizes are as follow:

1. Performance Across Graph Sizes (n=300, 400, 500): All algorithms consistently found the largest connected component (LCC) of equivalent size across the random graphs of increasing node counts (n=300, 400, 500). However, the custom heuristic generally achieves slightly higher maximum path lengths (Lmax), suggesting it might be more effective in finding longer paths within these graphs.
2. Consistency Across Algorithms: The results show that for smaller, randomized graphs (n=300, 400, 500), the average path lengths within the LCC and the sizes of the LCC remain stable across different heuristics, indicating that all heuristics perform similarly in terms of basic connectivity and path optimization metrics.
3. Comparison in Specialized Graphs: In the specialized graph datasets (DSJC500-5, Euroroad, and Power graph), the custom heuristic generally underperformed compared to DFS and Dijkstra in terms of Lmax on the Euroroad and Power graphs but matched their performance on the DSJC500-5 graph. This might indicate a need for further optimization of the custom heuristic for specific types of graphs or more complex connectivity patterns.
4. Efficiency on Large and Complex Networks: The significant reduction in Lmax by the custom heuristic in the Power graph (from 1038 to 511) suggests that the heuristic may be

more efficient or effective in handling very large and dense networks, potentially offering computational or performance advantages in specific scenarios.

In summary, while the custom heuristic often performs comparably to traditional heuristics and even excels in certain metrics on larger or more complex networks, there is a noticeable variance in effectiveness based on graph type and size. This variability suggests that while the heuristic is robust, its adaptability could be enhanced to consistently match or exceed the performance of established algorithms across all types of graphs. Further research and development might focus on optimizing the heuristic for specific graph characteristics to leverage its strengths more uniformly.

6. REFERNCES:

[1] T. H. Cormen, C. E. Leiserson, R. L. Rivest, and C. Stein, Introduction to Algorithms, 4th ed. Cambridge, MA: The MIT Press, 2022. [Online]. Available: https://mitpress.mit.edu/books/introduction-algorithms-fourth-edition [Accessed: Apr. 22, 2024].

[2] "Wikipedia contributors. 'A* search algorithm.' Wikipedia, The Free Encyclopedia. Wikipedia, The Free Encyclopedia, Web. [09-April-2024]. Available: https://en.wikipedia.org/wiki/A*_search_algorithm".